Deep CFR, explained

A pedagogical companion: from regret matching to a neural GTO poker player (with the practical lessons of the PokerTrainer project)

PokerTrainer documentation

June 2026

Abstract

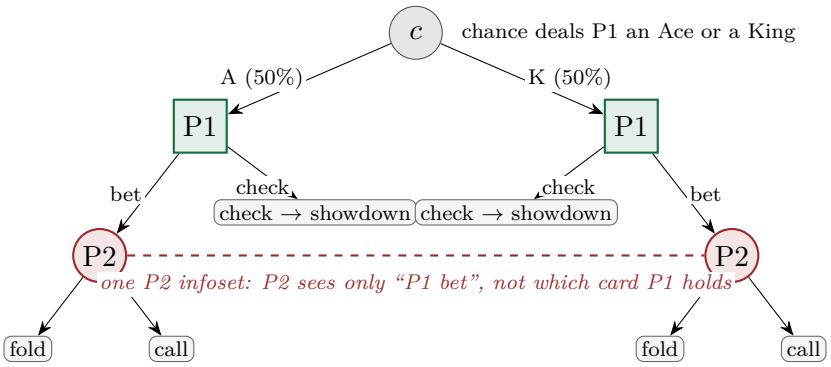
This document explains, from first principles, how one builds a near-GTO (Game-Theory-Optimal) poker agent with *Deep Counterfactual Regret Minimization* (Deep CFR, Brown et al. 2019). It is written to be read top-to-bottom by someone who knows Python and basic probability but has never touched game theory. Every concept comes with a diagram or Python pseudo-code, and the final sections summarize the modern landscape of poker AI (DeepStack, Libratus, ReBeL, Player of Games, AlphaHoldem) and the concrete engineering lessons learned while implementing Deep CFR in this repository.

Contents

1 The problem: playing well when you cannot see everything

Chess and Go are *perfect information* games: both players see the whole board. Poker is different — you see your own cards but not your opponent's. This single fact breaks the tools that work for chess (minimax search, AlphaZero-style self-play on states), because in poker **a "state" is not enough**: the best action depends on a *probability distribution* over what the opponent may hold, and that distribution depends on how both players have been playing. Strategies must be *randomized*: if you only raise with aces, observant opponents fold and your aces earn nothing.

Information set (infoset). All game states a player cannot tell apart. In heads-up poker, an infoset is: *my hole cards + the visible board + the betting history so far.* The opponent's hidden cards vary across the states inside one infoset, and a strategy must choose the *same* action distribution for the whole infoset — you cannot condition on what you cannot see.



Nash equilibrium / GTO. A pair of strategies such that neither player can gain by deviating unilaterally. In two-player *zero-sum* games (like heads-up poker), playing a Nash strategy guarantees you cannot lose in expectation, no matter what the opponent does. "GTO play" is the poker community's name for it. The distance from equilibrium is called **exploitability**: how much a perfect counter-strategy would win against you. Solving a game = driving exploitability toward 0.

Intuition

GTO is *defensive*: it does not try to exploit a weak opponent, it makes *you* unexploitable. That is exactly the right target for training: it is a fixed mathematical object we can measure progress against — unlike "beats opponent X", which moves whenever X changes.

2 Regret: the engine of everything

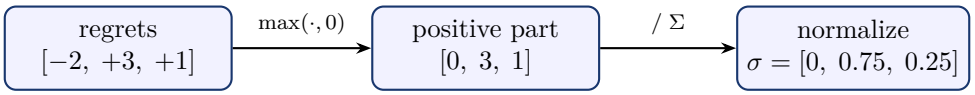
Everything in CFR is built on one simple, almost embarrassing idea: *keep track of how much you regret not having played each action, and play actions you regret in proportion to that regret.*

Let us make "regret" a number you could actually count. Fix *one* decision point, and suppose you have faced it 10 times so far, sometimes folding, sometimes calling, sometimes raising. Adding up those 10 outcomes, you won **5 chips in total**. Now replay history three times, each time asking one question: *what if I had played the same single action at every one of those 10 visits, with everything else unchanged?*

hindsight experiment	total chips	vs. the 5 you actually won	regret
"I always fold"	3	$3-5$	-2
"I always call"	8	$8-5$	$+3$
"I always raise"	6	$6-5$	$+1$

The **regret of an action** is exactly that last column: what the action *would* have earned you, minus what you *actually* earned. Positive regret (+3 for `call`) means "I wish I had played this more"; negative regret (−2 for `fold`) means "glad I didn't". Collecting the column into a vector, your cumulative regrets at this decision point are $R = [-2, +3, +1]$. **Regret matching** (Hart & Mas-Colell, 2000) converts these into a strategy — play each action in proportion to how much you regret not playing it:

$$\sigma(a) \;=\; \frac{\max(R(a),\,0)}{\sum_b \max(R(b),\,0)} \qquad \text{(if all regrets} \le 0\text{: play } \arg\max_a R(a)\text{)}.$$



Listing 1: Regret matching (the whole thing).

```python
def regret_matching(R, legal):              # R: regret per action
    pos = [max(R[a], 0.0) for a in legal]
    z = sum(pos)
    if z > 0:
        return {a: p / z for a, p in zip(legal, pos)}
    # all regrets non-positive: play the least-bad action
    best = max(legal, key=lambda a: R[a])    # (ties: split equally)
    return {a: 1.0 if a == best else 0.0 for a in legal}
```

Why is this magic? Because of a classical theorem: if you keep updating regrets and playing regret matching, your *average* regret per round goes to zero ($O(1/\sqrt{T})$). And in two-player zero-sum games there is a beautiful correspondence (the "folk theorem" of online learning):

If both players have average regret $\le \varepsilon$, the pair of *time-averaged* strategies is a 2ε-Nash equilibrium. Self-play regret minimization ⇒ convergence to GTO.

Intuition

Note carefully *which* object converges: the **average** of all the strategies you used across training, not the latest one. The current strategy oscillates forever (it keeps probing); the *average* of the oscillation is the equilibrium. This is why Deep CFR ends with a separate "average policy" network — it is the actual product of the whole procedure.

3 CFR: regret, but at every infoset of a game tree

Regret matching solves a single decision. A poker game is a *tree* of decisions. **Counterfactual Regret Minimization** (Zinkevich et al. 2007) states that you can run an independent regret matcher *at every infoset*, each fed with the right local quantity, and the global average strategy still converges to Nash. The right local quantity is the *counterfactual value*:

Counterfactual value $v(I, a)$: the expected payoff of playing action a at infoset I, weighted by the probability that the *others* (opponent and chance) play into I — as if *you* always tried to reach I. The **instantaneous regret** of a at I under the current strategy σ_t is $r_t(I, a) = v(I, a) - \sum_b \sigma_t(I, b)\, v(I, b)$, i.e. "action a versus what I currently do on average at I".

Vanilla CFR then loops: traverse the *whole* tree, accumulate $R_T(I, a) = \sum_t r_t(I, a)$ at every infoset, update every σ by regret matching, repeat. Here it is in full — the bookkeeping to watch is the

two *reach probabilities* carried down the tree, because each one is used for a different purpose at the update:

Listing 2: Tabular vanilla CFR (two players, chance sampled once per traversal). The asymmetry to internalize: regrets are weighted by the *opponent's* reach (that is what makes values "counterfactual"), while the average strategy is weighted by *your own* reach (you only average a decision where you would actually arrive).

```
R = defaultdict(zeros)      # cumulative regrets,  one vector per infoset
S = defaultdict(zeros)      # cumulative strategy, one vector per infoset

def cfr(state, t, reach):
    """reach[p] = probability that player p's OWN choices so far would
    lead them here. Returns (u_0, u_1), expected values under sigma_t."""
    if state.is_terminal():
        return state.payoffs()                  # zero-sum pair

    I, p  = infoset(state), state.to_act()
    sigma = regret_matching(R[I], state.legal())

    v, v_I = {}, [0.0, 0.0]
    for a in state.legal():
        r = list(reach)
        r[p] *= sigma[a]                        # my move scales MY reach
        v[a] = cfr(state.child(a), t, r)
        v_I[0] += sigma[a] * v[a][0]
        v_I[1] += sigma[a] * v[a][1]

    for a in state.legal():                     #  the two updates
        R[I][a] += reach[1 - p] * (v[a][p] - v_I[p])   # OPPONENT's reach
    S[I] += reach[p] * sigma                            # MY OWN reach
    return v_I

def solve(T):
    for t in range(1, T + 1):
        cfr(deal_random_hand(), t, reach=[1.0, 1.0])   # chance sampled
    return {I: S[I] / S[I].sum() for I in S}    # AVERAGE strategy ~ Nash
```

Two practical refinements matter for us — and both are tiny edits to the update lines above:

- **Linear CFR** (Brown & Sandholm 2019): weight iteration t's contributions by t. Early iterations are garbage (the strategy was random); weighting by t discounts them and speeds convergence a lot.
- **CFR$^+$** (Tammelin 2014): clip cumulative regrets at zero after every update, so an action can recover quickly once it stops being bad. (Used by the commercial GTO solvers and by Cepheus to essentially solve limit hold'em; Deep CFR uses Linear CFR instead.)

Listing 3: CFR$^+$ and Linear CFR are three-line edits to the two update lines.

```
#  vanilla CFR
R[I][a] += reach[1 - p] * (v[a][p] - v_I[p])
S[I]    += reach[p] * sigma

#  CFR+ : clip cumulative regret at 0; average weighted by t
R[I][a]  = max(R[I][a] + reach[1 - p] * (v[a][p] - v_I[p]), 0.0)
S[I]    += t * reach[p] * sigma
# (CFR+ also alternates: update one player's regrets per traversal)
```

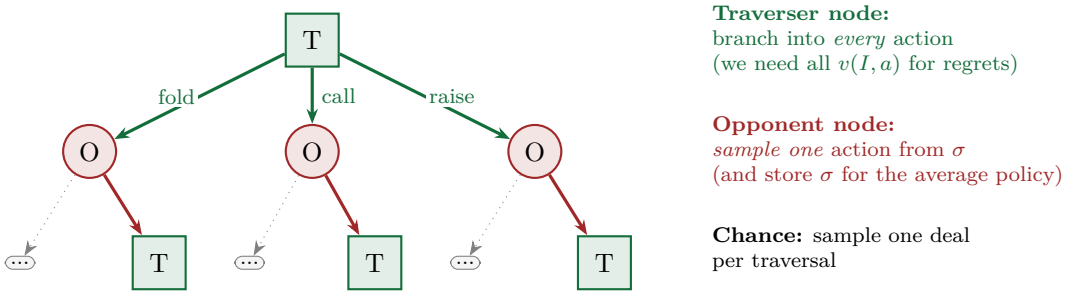
```
#  Linear CFR : weight EVERYTHING by the iteration number t
R[I][a] += t * reach[1 - p] * (v[a][p] - v_I[p])
S[I]    += t * reach[p] * sigma
# equivalent in Deep CFR: store plain samples tagged with t, and weight
# the regression loss by t  (exactly what trainer/cfr/train.py does)
```

Trap (learned the hard way in this project)

CFR's guarantee is about the *average* strategy. Looking at the *current* network/strategy mid-training and judging "it plays weird" is not, by itself, evidence of failure — but a current strategy frozen at uniform, or one that never discriminates hand strength, *is* (see Section 8).

4 Monte Carlo CFR: sample the tree instead of visiting it all

Heads-up no-limit hold'em has about 10^{160} states. Vanilla CFR visits the whole tree per iteration — impossible. **External-sampling MCCFR** (Lanctot et al. 2009) fixes this with one elegant asymmetry per traversal. One player is the *traverser* (we are computing *their* regrets this traversal):

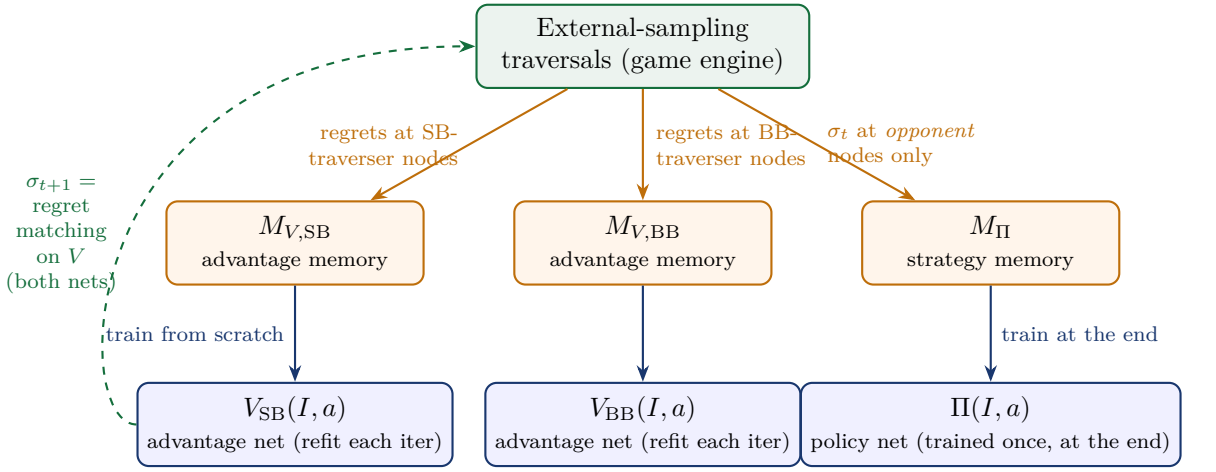


This sampling scheme has two crucial properties, both provable: (1) the sampled regret estimates are *unbiased*; and (2) opponent infosets are visited with probability proportional to the opponent's *own* probability of reaching them under σ — which is exactly the weighting needed when collecting strategy samples for the average policy (more on this in Section 5, it caused a real bug in this repo).

5 Deep CFR: replace the tables with networks

Even MCCFR keeps a *table* of regrets, one row per infoset — still impossibly many in no-limit poker. Historically this was handled with hand-crafted *abstraction* (bucketing similar hands together). **Deep CFR** (Brown, Lerer, Gross, Sandholm 2019) removes the tables and the abstraction: a neural network *generalizes* regrets across infosets.

5.1 The three memories and two kinds of networks



Π ***is the GTO player*** *(the average strategy)*

- **Advantage networks** $V_p(I, a)$, one per player: trained to predict the (linearly weighted, time-averaged) regret of each action at infoset I. The current strategy is obtained by *regret matching on the network's outputs*: $\sigma_t(I) = \mathrm{RM}(V_p(I, \cdot))$.
- **Advantage memories** $M_{V,p}$: reservoir buffers of $(I,\ t,\ \tilde{r}_t(I, \cdot))$ samples collected at the traverser's nodes.
- **Strategy memory** M_Π: reservoir buffer of $(I,\ t,\ \sigma_t(I))$ samples collected at *opponent* nodes only.
- **Policy network** Π: trained *once at the very end* on M_Π — it distills the time-averaged strategy, the thing that actually converges to Nash.

5.2 The algorithm, in Python pseudo-code

This is Algorithm 1 of the paper, written the way this repository implements it (`trainer/cfr/cfr_coro.py`):

Listing 4: One external-sampling traversal (the heart of Deep CFR).

```
def traverse(state, p, t):
    """Recursive traversal; returns the expected value for traverser p."""
    if state.is_terminal():
        return state.payoff(p)

    I     = infoset(state)                  # what the actor can see
    actor = state.to_act()
    legal = state.legal_actions()

    # current strategy at I = regret matching on the actor's advantage net
    sigma = regret_matching(V[actor](I), legal)

    if actor == p:                          #  TRAVERSER node
        v = {}
        for a in legal:                     # branch into EVERY action
            v[a] = traverse(state.child(a), p, t)
        v_I = sum(sigma[a] * v[a] for a in legal)
        regrets = {a: v[a] - v_I for a in legal}
        M_V[p].reservoir_add((I, t, regrets))
        return v_I
    else:                                   #  OPPONENT node
```

```
        M_Pi.reservoir_add((I, t, sigma))  # strategy sample (HERE only!)
        a = sample(sigma)                  # play ONE action
        return traverse(state.child(a), p, t)
```

Listing 5: The outer loop. Note the two unusual moves: networks are retrained *from scratch* each iteration, and the deliverable is the policy net trained at the end.

```
def deep_cfr(T, K):
    for t in range(1, T + 1):
        for p in PLAYERS:                          # SB, BB
            for k in range(K):                     # K traversals per player
                state = deal_random_hand()         # chance is sampled
                traverse(state, p, t)
            V[p] = AdvantageNet()                  # fresh net (from scratch!)
            fit(V[p], M_V[p],                      # regression on the memory
                loss=lambda pred, r, t_i: t_i * mse(pred, r))  # Linear CFR
    Pi = PolicyNet()
    fit(Pi, M_Pi, loss=lambda pred, sig, t_i: t_i * ce(pred, sig))
    return Pi                                      # the (near-)GTO player
```

Listing 6: Reservoir sampling (Vitter's Algorithm R): a fixed-size buffer that remains a *uniform* sample of everything ever inserted — this is what lets a finite memory represent all T iterations fairly.

```
def reservoir_add(buf, item):
    buf.n_seen += 1
    if len(buf) < buf.capacity:
        buf.append(item)
    else:
        j = randint(0, buf.n_seen - 1)             # uniform over the stream
        if j < buf.capacity:
            buf[j] = item                          # replace a random slot
```

5.3 Why each piece is the way it is

Intuition

Why predict regrets and not "the best action"? Because CFR's convergence machinery runs on regrets. The network is just a compressed, generalizing regret table: two infosets with similar cards and similar histories share statistical strength instead of each needing its own row.

Intuition

Why retrain from scratch each iteration? The target — the time-averaged regret — changes every iteration. Warm-starting from the previous net biases the regression toward stale targets; the paper found from-scratch refits more reliable. (It costs wall-clock time, which is why the refit budget is a key hyperparameter.)

Intuition

Why store strategy samples only at opponent nodes? External sampling makes the opponent play through its own nodes with probability equal to its own reach — so the stream of opponent-node σ samples is *automatically* reach-weighted, which is the exact weighting the average strategy needs. Traverser nodes are reached by brute branching (every action, regardless of σ), so including them would inject reach-*unweighted* samples and bias the final policy.

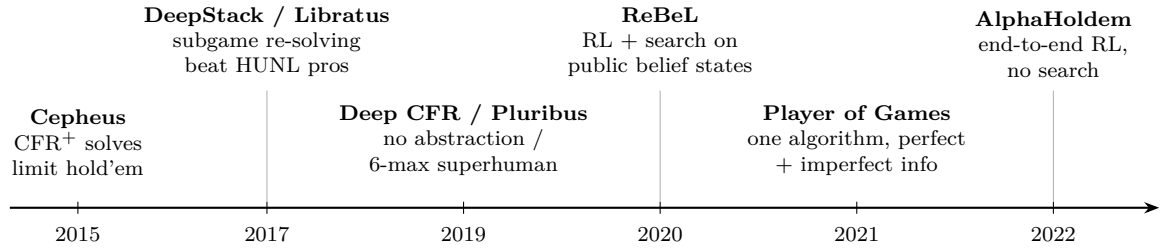
Trap (learned the hard way in this project)

This repository initially stored σ at *every* node "to collect more data". More data, wrong distribution: the PolicyNet — the actual output of the algorithm — was being trained on a biased average. Paper fidelity matters most precisely where it is least obvious.

5.4 Small but load-bearing details

- **All-non-positive regrets ⇒ argmax.** When $\sum_a \max(V(I,a),0) = 0$, the paper plays the highest-advantage action *deterministically*. Uniform would keep wasting opponent samples on known-bad actions.
- **Iteration 1 must be uniform.** A freshly initialized network outputs noise; regret matching on noise gives a degenerate one-hot strategy that can collapse one player's traversal trees. Fix used here: zero-initialize the advantage net's output head, and make the argmax fallback split exact ties equally — all-zero outputs then tie everywhere and degrade gracefully to uniform, with no special case in the traversal.
- **Linear weighting lives in the loss.** Each sample carries its iteration t; the regression weights samples by t. This implements Linear CFR without ever touching stored targets.

6 The modern landscape: ways to build a GTO player



Three families of approaches, with different trade-offs:

1. Tabular CFR on an abstraction + subgame re-solving (2015–2017). Bucket strategically similar hands, solve the smaller game with CFR$^+$/MCCFR (the "blueprint"), then at play time re-solve the current subgame more finely (*Libratus*: nested safe subgame solving; *DeepStack*: continual re-solving with learned counterfactual-value networks as depth limits). Strong but engineering-heavy; the abstraction is hand-crafted.

2. Neural CFR without search (Deep CFR, 2019; this project). Let a network generalize across infosets; no abstraction, no play-time search — the policy net plays directly. Conceptually clean, single training loop; the price is regression noise on regrets, so per-game exploitability is higher than search-based systems. *Pluribus* (6-max superhuman, 2019) is the hybrid cousin: an MCCFR blueprint plus depth-limited re-solving at play time.

3. RL + search on belief states (2020–). *ReBeL* casts the imperfect-information game as a continuous-state perfect-information game over *public belief states* (distributions over private cards), and trains value/policy nets by self-play with CFR used inside the search. *Player of Games* unifies this with AlphaZero-style search so one algorithm plays chess, Go *and* poker. At the other extreme, *AlphaHoldem* drops search entirely (pure actor-critic end-to-end) and is much cheaper at play time, at some cost in robustness guarantees.

System	Year	Core idea	Search at play time?	Nets
Cepheus	2015	CFR$^+$, essentially solved limit HU	no	—
DeepStack	2017	continual re-solving + CFV nets	yes	value
Libratus	2017	abstraction blueprint + safe re-solving	yes	—
Deep CFR	2019	networks replace regret tables	no	value+policy
Pluribus	2019	blueprint + depth-limited search (6-max)	yes	—
ReBeL	2020	RL+search on public belief states	yes	value+policy
Player of Games	2021	sound search for both info regimes	yes	value+policy
AlphaHoldem	2022	end-to-end actor-critic, no search	no	policy

Intuition

The recurring trade-off is **search at play time**. Search buys lower exploitability per unit of training but costs latency, complexity, and (in belief-state methods) a tractable public state. Search-free methods (Deep CFR, AlphaHoldem) give you a single network you can ship anywhere — which is why this project chose Deep CFR as its backbone.

7 This project: Deep CFR with a token transformer and a curriculum

7.1 Encoding the infoset as a token sequence

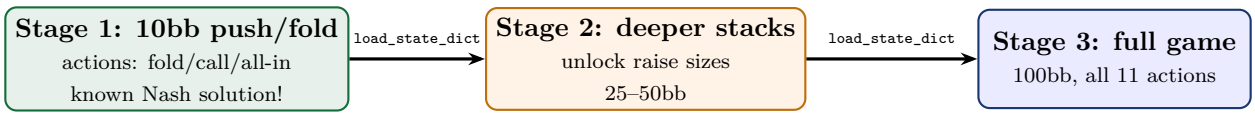
Early versions encoded the infoset as one flat 816-dimensional vector; the hole cards were 52 of those dimensions and the trained network simply *ignored them* (measured: the strategy varied by $< 2\%$ across all 38 test hands). The fix was structural, not capacity: encode the infoset as a **sequence of tokens** — each card, action, pot-size bucket and stack-size bucket gets its own token — and use a small transformer. Attention must then handle each card as a first-class object that cannot be averaged away into 700 other dimensions.

```
[BOS] [POS_SB] [Ah] [As] [SEP_PRE] [HERO][R100][POT_2][STACK_15]
                 [VILLAIN][AI][POT_9][STACK_0] [DECISION]
```

The network reads the hidden state at the final `[DECISION]` token and outputs one number per action (predicted regrets for the advantage net, logits for the policy net). The same architecture serves both nets.

7.2 Curriculum: solve an easy poker first

Training the full 100bb game directly kept producing hand-agnostic strategies (a chicken-and-egg: a hole-blind strategy generates hole-blind regret targets, so the net correctly learns a constant function and nothing improves). The project's answer is a **curriculum over the game itself**, keeping the network's shape fixed across stages so weights transfer with a plain `load_state_dict`:



Stage 1 is special: 10bb heads-up push/fold has a *known* Nash equilibrium. The project computes that equilibrium itself (Monte-Carlo equity matrix over the 169 starting-hand classes + fictitious play), cross-checks it against published charts (≈90–96% agreement, boundary hands only), and then **validates training online**: every iteration, the trainer reads the net's jam/call frequency

for all 169 hands and prints a 13×13 grid plus an agreement score against the oracle. Learning is visible as the grid's frontier sharpening toward the chart — a ground-truth training signal that the full game simply does not offer.

7.3 Concept-to-code map

Paper concept	This repository
External-sampling traversal	`trainer/cfr/cfr_coro.py::traverse_coro`
Advantage / strategy memories	`trainer/cfr/buffers.py` (Algorithm R)
Regret matching (+ argmax fallback)	`trainer/cfr/regret_matching.py`
Advantage net refit (from scratch, t-weighted)	`trainer/cfr/train.py::refit_adv_net`
Policy net (the GTO player)	`trainer/cfr/train.py::train_policy_net`
Infoset encoding (tokens)	`trainer/cfr/tokenize.py`
Stage-1 Nash oracle + validation	`trainer/evaluate/pushfold_solver.py`, `trainer/cfr/pushfold_validation.py`

8 Practical lessons (what actually goes wrong)

These are the failures actually hit while building this system. Each one looked like "the algorithm doesn't work" and was something else.

Trap (learned the hard way in this project)

Weight decay silently kills weak signals. With Adam + `weight_decay=1e-3`, the advantage net froze into a *constant* strategy (same action for AA and 72o). Reason: hand-strength discrimination is the *weak* direction in the loss landscape; L2's fixed per-step shrink overwhelms it while the strong "always do X" direction survives. A controlled experiment showed `wd=0` learns the correct hand-dependent range and `wd=1e-3` learns none of it. Default to no weight decay for regret regression.

Trap (learned the hard way in this project)

Adam and regret matching are scale-invariant — so "rescaling the targets" cannot fix learning. Dividing all regret targets by 100 vs by 10 produced *byte-identical* results: regret matching is invariant under positive scaling, and Adam normalizes per-parameter gradient magnitude. The only thing target scale interacts with is weight decay (a fixed absolute pull). Lesson: before attributing an effect to a scale change, check what in the pipeline is actually scale-*sensitive*.

Trap (learned the hard way in this project)

Index-vs-action confusion across an API boundary. `env.step(i)` interpreted `i` against the engine's full legal list, while the curriculum-restricted traversal sampled `i` from a *filtered* list — so "all-in" silently executed as a min-raise, in 89% of traversals. No exception, no warning, just corrupted training data. Lesson: pass *actions* across boundaries, never bare indices; and write tests that inspect the *recorded history*, not just your own bookkeeping.

Trap (learned the hard way in this project)

Self-play loss curves cannot be trusted. Every collapse in this project came with beautifully decreasing losses — a hole-blind strategy generates hole-blind targets, which the net fits perfectly. The metrics that caught the failures were *external*: play probes against fixed opponents (does AA out-earn 72o?), per-hand strategy grids, and at 10bb the Nash oracle agreement score. Always validate against something the training loop cannot collude with.

9 Further reading

- Zinkevich, Johanson, Bowling, Piccione (2007). *Regret Minimization in Games with Incomplete Information* (CFR).
- Lanctot, Waugh, Zinkevich, Bowling (2009). *Monte Carlo Sampling for Regret Minimization in Extensive Games* (external sampling).
- Bowling, Burch, Johanson, Tammelin (2015). *Heads-up limit hold'em poker is solved* (Cepheus, CFR^+).
- Moravčík et al. (2017). *DeepStack: Expert-level artificial intelligence in heads-up no-limit poker.*
- Brown, Sandholm (2018). *Superhuman AI for heads-up no-limit poker: Libratus.* (2019): *Pluribus*, six-player.
- Brown, Lerer, Gross, Sandholm (2019). *Deep Counterfactual Regret Minimization* — the paper this document explains.
- Brown, Bakhtin, Lerer, Gong (2020). *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games* (ReBeL).
- Schmid et al. (2021). *Player of Games.*
- Zhao et al. (2022). *AlphaHoldem: High-Performance Artificial Intelligence for Heads-Up No-Limit Poker via End-to-End RL.*